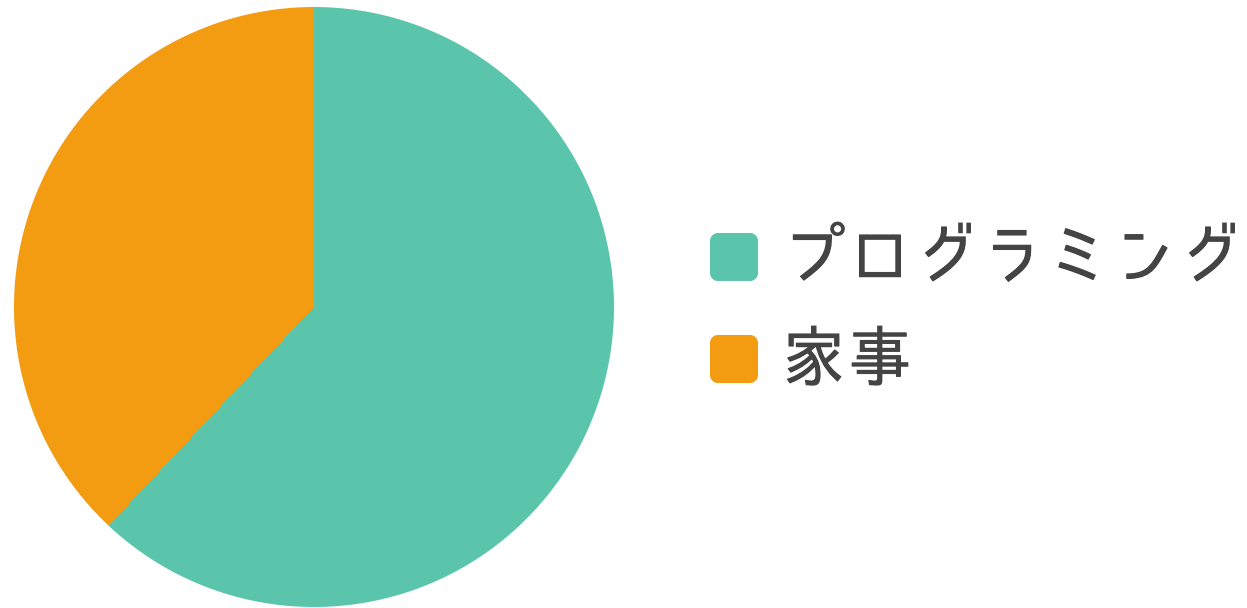


Claude Code をオーケストレーションして 自動でレビュー FB ループを回して 洗い物をしてる話

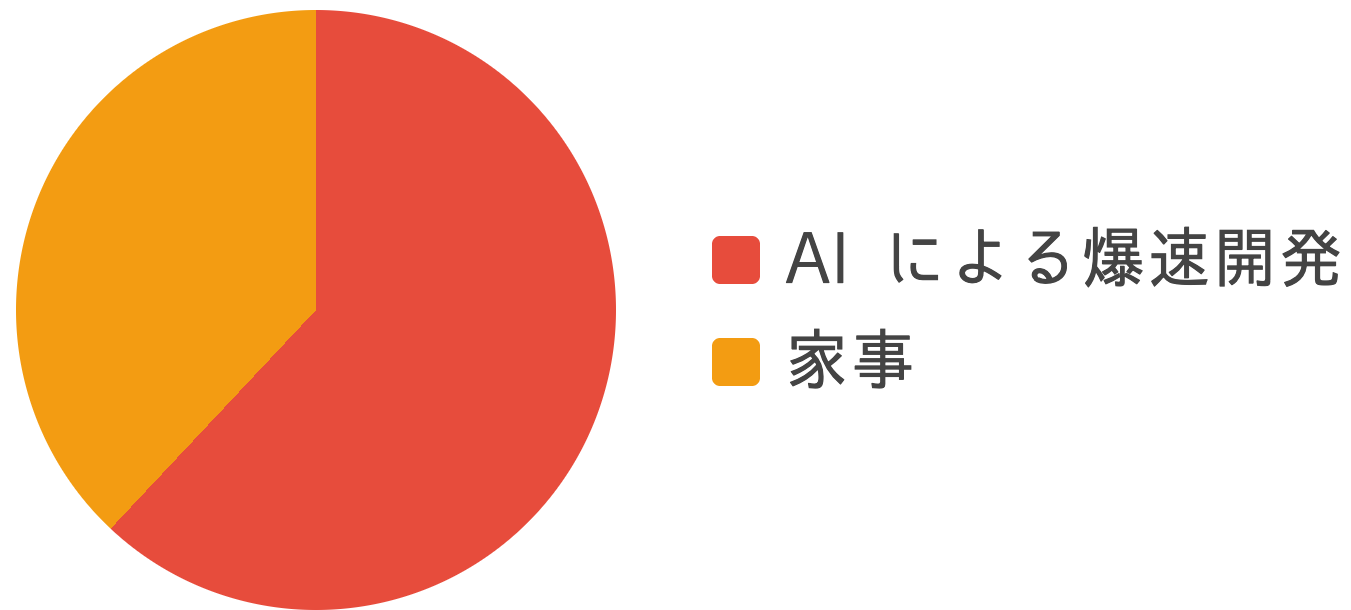
nrs / 成瀬允宣

みなさん、生活のリズムはどんな感じですか？

これまで

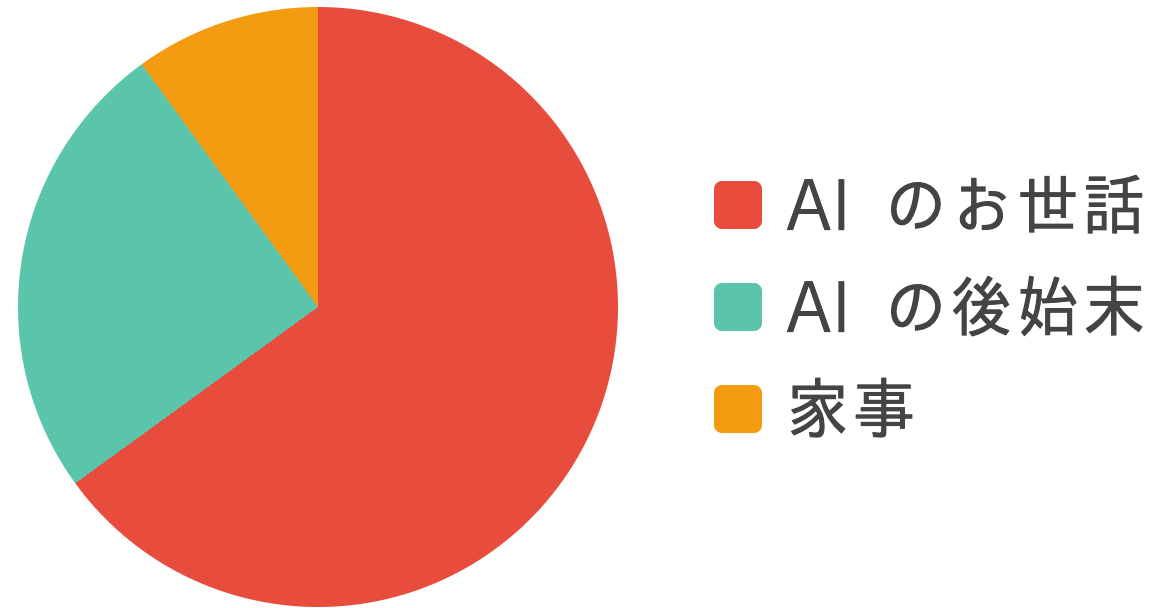


AI パラダイムシフト後（理想）



AI がサポートしてくれるから
プロダクト開発が捗るはず……

現実



家事ができない

洗い物や洗濯物が溜まってしまふ……

「洗い物がしたい」

洗い物をする技術

なぜ洗いができないのか

AI の出力を逐一チェックしなければならない

そもそもまともなコードを出してくれない

- フォールバックしまくり — 動けばいいという場当たりコード
- 謎の後方互換サポート — 誰も頼んでいない互換レイヤー
- DRY にならない — 同じロジックがコピペで増殖する
- etc.etc.etc.etc.etc.etc.
- 結局、人間が全行レビューしないと使えない
 - 将来レビュー不要になるかもしれんがいまはまだ……

要件と違う方向に進む

気づいたら別のものができている

- 指示を解釈違いして、関係ない機能を作り始める
- 途中で軌道修正するも、既に入った変更の巻き戻しが大変
- そもそも要件ができていない
- 「これじゃない」を伝えるコストが高い

止まる・許可を求め続ける

作業のたびに人間が呼ばれる

- かと思ったら止まってる — エラーで黙って停止
- 「許可申請」がでてくる
 - bypass しろって？ それはうーん……
- 1回の作業で何度も中断される

気づけば一日中 AI の相手をしている

AI の面倒を見るのが仕事になっている

- チェック、修正指示、承認、軌道修正の繰り返し
- 自分の作業に集中できる時間がない
- 家事をやる余裕など生まれるはずがない
- というか脳が焼ききれそう

私たちは AI のベビーシッターだ

どうすべきか？

選択肢 1: 子守をやめる

ひどいコードが量産される

- 動くけど保守できないコード
- 要件と違う実装が紛れ込む
- 不自然なハックだらけ

選択肢 2: AI を使わない

今後のキャリアに影響しそう…

- 周りにはどんどん AI を活用している
- 生産性の差が開いていく
- AI 時代に取り残されるリスク

選択肢 3: 畑をたがやす

「この職業に未来はない」と見切りをつける

- プログラミングは AI に取って代わられる前提
- 農業やまったく別の分野に転身する
- でも本当にそこまでの話だろうか？

本当にこれしかない？

ウルトラ C

AI に子守をさせよう

どんな子守？

最大の問題は何か

コードの品質が低すぎる

いま子守でやっていること

コードレビュー

- 実装が要件どおりにできているか
- 不自然なハックが入り込んでいないか

これを AI にやらせよう

なぜ「分離」が必要なのか

Lost in the Middle / Context Rot

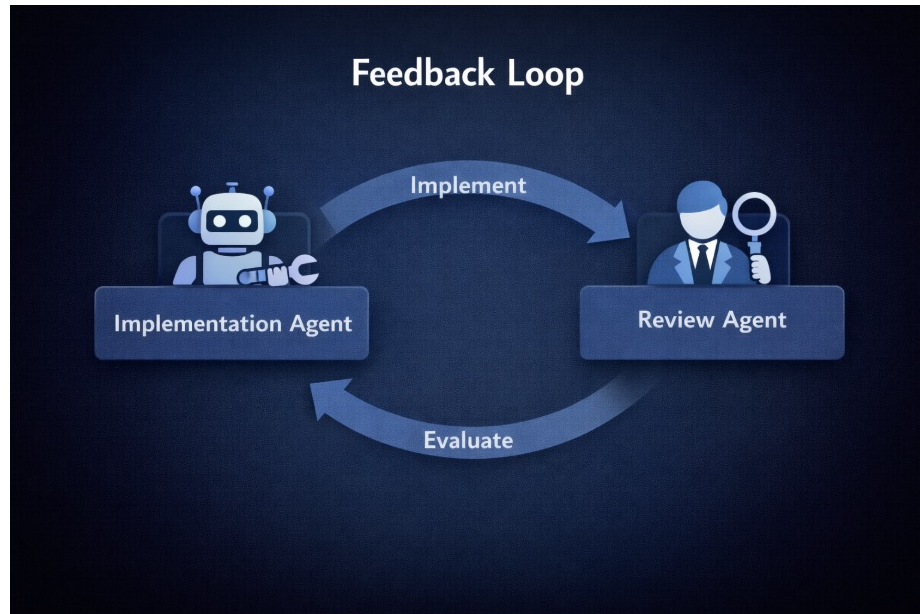
- LLM はコンテキストが長くなると中盤の情報を忘れる (Lost in the Middle)
- 長時間の作業でコンテキストが汚染される (Context Rot)
- 実装しながらレビューも兼ねると、どちらも中途半端になる

AI もマルチタスクが苦手

タスクを絞ることで精度が上がる

- 1エージェント1責務 — 実装だけ、レビューだけ
- コンテキストを分離すれば腐らない
- 人間のコードレビューと同じ — 書いた本人はバグに気づけない

Claude Code にレビューさせてみる



書く AI とレビューする AI を分離

人間がレビューする代わりに
Claude Code にレビューさせる

AI 同士でフィードバックループを回す

どのように実現すればよいのだろうか

Skill でやってみた

が簡単じゃなかった

- Skill で定義しても compaction で忘れる
- プロンプトで指示しても AI が手順をスキップする
- ユースケースごとにやるべきことが変わる

必要なのは
「強制力」と「再現性」

強制力はプログラムで確保する

手順をコードで制御すれば AI は飛ばせない

- 実装→レビュー→修正の状態遷移をプログラムで管理
- レビューを通過しない限り完了できない
- AI が勝手にステップを飛ばせない構造

再現性は YAML で確保する

同じ定義から同じプロセスが走る

- ワークフローを YAML で宣言的に定義
- 誰が実行しても同じ手順が再現される
- プロンプトではなく構造で制御する

→ これを**「宣言的オーケストレーション」**と命名

Step 1: ワークフローを YAML で定義する

```
movements:  
  - name: implement    # 実装する  
  - name: review       # レビューする  
  - name: fix           # 修正する
```

手順をステップとして宣言する。プロンプトではなく構造。

Step 2: レビューに強制力を持たせる

```
- name: review
  persona: reviewer
  edit: false           # コードを編集できない
  rules:
    - condition: 問題なし
      next: COMPLETE
    - condition: 修正が必要
      next: fix         # fix に強制遷移
```

edit: false でレビューアの改ざんを防ぎ、ルールで次の手順を強制する。

Step 3: 合格するまで繰り返す

```
- name: fix
  persona: coder
  rules:
    - condition: 修正完了
      next: review      # review に戻る
```

fix は必ず review に戻る。レビューを通過しない限り完了できない。

これが最高だった

何が最高だったのか

- レビューを飛ばせない
 - 品質が落ちない
- レビュー観点を作れる
 - フォールバックや後方互換を何度も指摘
- 同じ定義で何度でも再現できる
 - 毎回組み立て直さなくていい
- 放っておいてもループが回り続ける
 - 目を離せる
- 自走性が高い
 - タスクが複雑だと30分～2時間ほっとける

洗い物できた 🙌



コードを書いている間に
台所がきれいになっている。
これが真の自動化。

ありがとうございました

え？みんなもやりたい？

では洗剤とスポンジを買ってきてもらって……

TAKT

TAKT をインストールしよう

このオーケストレーションの仕組みは公開されています

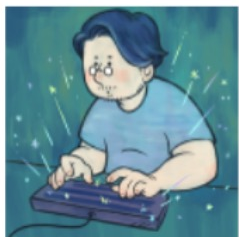


```
npm install -g takt
```

インストールしたその瞬間から
ビルトインで私と同じ体験ができます

←QRはユーザーコミュニティです
discord.gg/emy8b9dxNK

Profile



nrs (成瀬 允宣)

@nrslib

プログラマ
なるセミ 代表 / 株式会社コドモン

趣味：カンファレンス講演
学生支援
小学校支援
写真

